21th Parallel Programming Marathon
SSCAD – 2026

Calebe P. Bianchini[1] **and Italo Assis**[2]

[1]Mackenzie Presbyterian University

[2]Federal University of the Semi-Arid Region

Rules for Remote Contest

For all problems, read the input and output session carefully. For all problems, a sequential implementation is provided, and it is against the output of those implementations that the output of your programs will be compared to decide if your implementation is correct. You can modify the program in any way you see fit, except when the problem description states otherwise. You must upload a compressed file (*zip*) containing your source code, the *Makefile*, and an execution script. The script must have the name of the problem. You can submit as many solutions to a problem as you want. Only the last submission will be considered. The *Makefile* must have the rule all, which will be used to compile your source code. The execution script runs your solution the way you design it – it will be inspected to ensure it does not corrupt the target machine.

The execution time of your program will be measured running it with time program and taking the real CPU time given. Each program will be executed at least three times with the same input, and the mean time will be taken into account. The sequential program provided will be measured in the same way. You will earn points in each problem, corresponding to the division of the sequential time by the time of your program (speedup). The team with the most points at the end of the marathon will be declared the winner.

This problem set contains 4 problems; pages are numbered from 01 to 08.

General Information

Compilation

You should use **CC** or **CXX** inside your *Makefile*. Be careful when redefining them! There is a simple *Makefile* inside the problem package that you can modify. Example:

```
FLAGS=-O3
EXEC=sum
CXX=g++

all: $(EXEC)

$(EXEC):
	$(CXX) $(FLAGS) $(EXEC).cpp -c -o $(EXEC).o
	$(CXX) $(FLAGS) $(EXEC).o -o $(EXEC)
```

Each judging machine has its group of compilers. See them below and choose well when writing your *Makefile*. The compiler that is tagged as *default* is predefined in **CC** and **CXX** variables.

machine	compiler	command
host	GCC 9.4.0 (*default*)	C = gcc C++ = g++
MPI	Open MPI 4.1.1 (*default*)	C = mpicc C++ = mpic++
	GCC 9.4.0	C = gcc C++ = g++
gpu	NVidia CUDA 12.0.1 (*default*)	C = nvcc C++ = nvcc
	GCC 8.3.1	C = gcc C++ = g++

Submitting

General information

You must have an execution script that has the same name of the problem. This script runs your solution the way you design it. There is a simple script inside you problem package that should be modified. Example:

```
#!/bin/bash
# This script runs a generic Problem A
# Using 32 threads and OpenMP
export OMP_NUM_THREADS=32
OMP_NUM_THREADS=32 ./sum
```

Submitting MPI

If you are planning to submit an MPI solution, you should compile using *mpicc*/*mpic++*. The script must call *mpirun*/*mpiexec* with the correct number of processes (max: 160).

```
#!/bin/bash
# This script runs a generic Problem A
# Using MPI in the entire cluster (4 nodes)
mpirun -np 4 ./sum
```

Comparing times & results

In your personal machine, measure the execution time of your solution using *time* program. Add input/output redirection when collecting time. Use *diff* program to compare the original and your solution results. Example:

```
$ time -p ./A < original_input.txt > my_output.txt
real 4.94
user 0.08
sys 1.56

$ diff my_output.txt original_output.txt
```

Do not measure time and **do not** add input/output redirection when submitting your solution - the *auto-judge system* is prepared to collect your time and compare the results.

Problem A
Terrain Flow and Watersheds

Miguel Felipe Silva Vasconcelos

A digital terrain model is a rectangular grid of elevations. Water flows from each valid cell to one of its eight neighboring cells, provided that the destination has a strictly lower elevation. A cell with no strictly lower valid neighbor is a *sink*. The resulting directed paths determine how much upstream terrain contributes to each cell and which cells belong to the same watershed.

Your task is to **parallelize the supplied sequential implementation** of terrain flow analysis, reducing execution time while preserving its exact behavior and output. The computation consists of three results: flow directions, flow accumulation, and watershed labels.

For each valid cell, select the *lowest-elevation* valid neighbor among the eight surrounding positions, but only if that neighbor is strictly lower than the current cell. This is a comparison of elevations, *not* a distance-normalized slope calculation. Ties must be resolved exactly as in the reference implementation, including its edge-specific order of neighbor traversal.

The direction codes are N$=1$, NE$=2$, E$=3$, SE$=4$, S$=5$, SW$=6$, W$=7$, and NW$=8$; a sink has direction 0. North means the preceding grid row. Cells whose elevation equals the no-data value are excluded from the flow network: they cannot send or receive flow and retain direction -9 and accumulation -1.

The *flow accumulation* of a valid cell is one (the cell itself) plus the accumulations of all immediate upstream neighbors whose directions point into it. Each valid cell also receives a *watershed label*: two cells share a label if following their flow directions reaches the same terminal sink. These are sink-defined basins, not catchments of externally selected outlets.

Watershed labels begin at 101 and are assigned by the first encounter of each basin in row-major traversal order, *not* by the sink's position. Missing cells retain label -1. The number of cells in each watershed must equal the flow accumulation at its sink. Flow accumulation and watershed labeling both depend on directions, but labeling does not depend on accumulation.

Input

The executable receives one terrain-file path. The file starts with six whitespace-separated numeric values, without field names:

```
rows columns xorigin yorigin cellsize nodata
```

The first five values are integers. `nodata` and the elevations are C++ `float` values. The origin coordinates and cell size are metadata and do not affect the computation. There must be at least one row and two columns. The header is followed by exactly `rows` $\times$ `columns` finite elevation values in row-major order.

The input must be read from the standard input.

Output

The program must reproduce the sequential executable's standard output *exactly*, including whitespace and numeric formatting. It prints the following sections, in order:

1. `the terrain`, followed by the original elevation grid, without ghost rows;
2. `the directions`, followed by the direction grid, including an extra top ghost row filled with 1 and an extra bottom ghost row filled with 5;
3. `flow accumulation` (including its trailing space), followed by the accumulation grid, including top and bottom ghost rows filled with 0;
4. `watersheds`, followed by the watershed-label grid, including top and bottom ghost rows filled with -1.

The direction, accumulation, and watershed grids therefore each contain `rows + 2` rows, while the terrain grid contains `rows` rows. No-data cells retain their designated values. Preserve the reference implementation's deterministic tie-breaking, watershed numbering, and formatting.

The output must be written to the standard output.

Example

Sample Input 1	Sample Output 1
6 6 0 0 1 -9999 78 72 69 71 58 49 74 67 56 49 46 50 69 53 44 37 38 38 64 58 55 22 31 24 68 51 47 21 16 19 74 53 54 12 11 12	the terrain 78 72 69 71 58 49 74 67 56 49 46 50 69 53 44 37 38 38 64 58 55 22 31 24 68 51 47 21 16 19 74 53 54 12 11 12 the directions 1 1 1 1 1 1 4 4 4 4 5 6 4 4 4 5 6 5 3 3 4 5 6 5 4 2 4 4 5 6 3 3 4 4 5 6 2 2 3 3 0 7 5 5 5 5 5 5 flow accumulation 0 0 0 0 0 0 1 1 1 1 1 1 1 2 2 2 4 1 1 3 7 9 1 2 1 1 1 18 1 3 1 4 6 2 23 1 1 1 1 8 36 1 0 0 0 0 0 0 watersheds -1 -1 -1 -1 -1 -1 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 101 -1 -1 -1 -1 -1 -1

Problem B
Seismic Pulse

Matheus Serpa

In a laboratory test, a homogeneous rock block receives a point shot and the response of the block is followed over time. Before the shot, the block is not at rest: small random vibrations, the background noise, are present throughout its interior. The pressure field $u(x, y, z, t)$ obeys the wave equation

$$\frac{\partial^2 u}{\partial t^2} = c^2 \, \nabla^2 u, \tag{1}$$

with velocity $c = 2500$ m/s.

The block is represented by a cubic grid of $N \times N \times N$ points, with spacing $h = 12.5$ m, and time advances in steps of $\Delta t = 1$ ms. Each point has integer coordinates (i, j, k), from 0 to $N - 1$. The 6 outermost layers of points on each face form a frame where the pressure is always zero; the remaining points are the interior ones.

The operator L applies, along each axis, the 12th-order centered difference, which uses 6 neighbors on each side with the Taylor coefficients, so that $L\,u \approx h^2 \, \nabla^2 u$. With $s = (c \, \Delta t / h)^2$, each step approximately solves the system

$$\left(I - \tfrac{s}{2} \, L\right) u^{n+1} = 2\,u^n - u^{n-1} + \tfrac{s}{2} \, L\,u^n \tag{2}$$

through a single Gauss-Seidel sweep over the pressure array itself, visiting the interior points with i varying fastest, then j, and finally k. When a point is updated, neighbors already visited by the sweep contribute their new value u^{n+1}, and those not yet visited contribute u^n.

Initially, each interior point receives a uniform pseudo-random value between -1 and 1, the same for u^{-1} and u^0. The shot happens at point $(40, 48, 56)$: at every step, right after that point is updated, one sample of a 25 Hz Ricker pulse is added to it.

Your task is to parallelize the provided sequential program. The output of the parallel version must match the output of the sequential program byte for byte. Use the compilation options of the provided Makefile. Keeping `-ffp-contract=off` is mandatory: without it, the compiler may fuse floating-point multiplications and additions, changing the results.

Input

A single line contains two integers, N and T ($80 \leq N \leq 480$, $1 \leq T \leq 160$): the grid size and the number of simulated time instants, counting the initial one. The program therefore performs $T - 1$ steps.

The input must be read from the standard input.

Output

First, the final values of u along the line $j = 48$, $k = 56$, with increasing i, only those whose absolute value lies between 0.0005 and 500, inclusive, one per line, with six decimal places. Last, a line with the sum of all N^3 final values in double precision: each plane k is summed with i varying fastest and then j, and the plane sums are accumulated in increasing k order. This sum is printed with the C format `%.17g`.

The output must be written to the standard output.

Example

Input	Output
84 3	0.310539 0.570899 0.311929 -0.064797 -0.017490 0.076555 -0.554166 -0.625080 0.010082 -0.158154 0.006525 -0.396908 -0.088861 -0.049836 0.144818 0.001903 -0.007060 -0.200129 -0.000812 -0.059112 -0.308306 0.141954 0.329577 -0.199208 -0.411873 -0.421019 0.078902 0.290722 0.270541 0.416854 0.246687 -0.057659 -0.145396 0.177555 -0.351090 -0.209131 -0.176865 0.126306 0.367907 0.011417 -0.058757 0.254176 0.106818 0.071088 0.342754 0.106163 0.023566 -0.128577 0.107477 0.018403 0.268726 0.352839 0.094108 -0.127990 -0.424255 -0.133630 -0.249489 0.020393 -0.103478 0.197927 0.080844 0.124039 -0.081922 0.295457 -0.024057 -0.112752 -0.078422 -0.314658 -0.157350 -0.198623 0.168072 -0.108563 13.331994413740176

In the example, values are separated by spaces to save room; in the actual output, each value is on its own line.

Problem C
Quadtree

Matheus Serpa

A quadtree is a hierarchical data structure for organizing points in a two-dimensional space. Each node represents a rectangular region, which may be recursively divided into four quadrants: northwest (NW), northeast (NE), southwest (SW), and southeast (SE). Dense regions are subdivided further, while sparse regions remain represented by larger nodes.

In the provided sequential C program, points are generated inside a rectangular domain and inserted into a quadtree. Each leaf stores at most four points. Inserting a fifth point causes the leaf to be subdivided and its existing points to be redistributed among the children. Subdivision stops when the implementation's minimum-size tolerance (`MAX_ERROR`) is reached.

After construction, the program searches for all points within a rectangular query region. Nodes whose regions do not intersect the query can be skipped, an optimization known as *spatial pruning*. Query results are stored in a singly linked list and written to a binary file. The program also releases its dynamically allocated data structures.

Your task is to **parallelize the provided sequential implementation**, reducing execution time while preserving its behavior and the required results. The reference implementation, including point generation, boundary handling, insertion, query traversal, and binary serialization, defines the expected behavior. The random number generator is initialized with `srand(42)` to make runs reproducible.

Input

Input

The input consists of a single test case. The parameters `x_min`, `x_max`, `y_min`, and `y_max` define the spatial domain. The four parameters prefixed with `sub_` define the rectangular query region. The parameter `num_points` specifies how many pseudorandom points are generated.

Point coordinates use the domain bounds and the reference generator. Rectangle membership includes the boundaries.

The input must be read from the standard input.

Output

The program must write the points returned by the spatial query to standard output. Each point consists of two coordinates (`x` and `y`), printed in the reference implementation's result order.

The output must preserve the reference implementation's formatting, including coordinate precision, separators, and line breaks. Do not add extra headers, delimiters, or a point count unless the reference program does so.

The parallel implementation must preserve the reference program's observable behavior, including the query results, their order, and the exact textual output.

The output must be written to the standard output.

Examples

Sample Input 1	Sample Output 1

`x_min = -10000`
`x_max = 10000`
`y_min = -10000`
`y_max = 10000`
 	
`sub_x_min = -500`
`sub_x_max = 501`
`sub_y_min = -502`
`sub_y_max = 503`
 	
`num_points = 1000`
	`(185.0,-148.6)`